

Ahsanullah University of Science & Technology
Department of Computer Science & Engineering



A ludo game

Computer Graphics Lab (CSE 4204)

Final Project Report

Submitted By:	
Sharadindu Dutta	20200104037
Kalvin Chakma	20200104038
Section and Group	A2 , Group 3 (Project 7)

1. Problem Analysis

1.1 Project Requirements

The goal of this project is to build a fully playable, visually polished 3D Ludo scene rendered with Three.js. The two required objects are the Ludo board and the Ludo dice, and both are rendered using a custom lit shader rather than a built-in Three.js material, so that lighting, specular highlights and a fresnel rim effect can be controlled directly. The board's default skin is an external image texture (ludo.jpg — a photograph of a traditional Ludo board) loaded with THREE.TextureLoader; clicking the board cycles it through four additional canvas-drawn procedural skins (Classic, Neon, Pastel, Walnut) before returning to the original image. The dice uses six procedurally textured pip faces (1–6, with opposite faces summing to 7), drawn entirely on canvas at runtime.

Beyond the two required objects, the scene includes 16 decorative pawns (four per player colour — Red, Green, Yellow, Blue) built from primitive Three.js geometries (cylinders and a sphere) and lit with a standard PBR material, and a circular wooden table whose surface uses an externally loaded wood-plank photograph (background.png) as its texture, in addition to full keyboard and mouse interactivity and a physically-plausible dice roll animation.

1.2 Core Integration

The board and dice geometry are built directly with Three.js primitives. The dice's six pip faces and the four alternate board skins are textured using HTML canvas drawing — drawn onto a canvas which is then used as a texture map. The board's default surface and the table underneath instead use externally loaded images (ludo.jpg for the board, background.png for the table) as their texture maps, loaded once at startup with THREE.TextureLoader. A PerspectiveCamera is combined with an orbit-style rig driven by keyboard input so the player can view the board from any angle. Lighting is integrated in two places at once: standard Three.js light objects (AmbientLight, a shadow-casting DirectionalLight acting as the sun, and a PointLight accent) illuminate the scene normally, and their color/intensity/position values are also passed as uniforms into the custom shaders, so the hand-written lighting model inside the shader stays consistent with the scene's actual lights. The decorative pawns and the table use Three.js's built-in MeshStandardMaterial (PBR) rather than the custom shader, and still receive the same scene lights.

1.3 Technical Challenges

The main mathematical challenge was the dice roll animation. Instead of snapping the dice to a random face, the roll is implemented as a tumble phase (compounded angular velocity across multiple axes to simulate a physical toss) followed by a settle phase, where the dice's current orientation is interpolated with spherical linear interpolation (slerp) onto the quaternion that faces the chosen result upward. This keeps the rotation visually smooth and avoids the shortest-path artifacts that plain Euler-angle interpolation would introduce. The custom shaders

themselves implement a manual Blinn-Phong lighting model in the fragment shader (diffuse + specular terms computed from the light uniforms and surface normal), a fresnel term for the rim highlight, and a small animated pulse driven by elapsed time, all of which required passing the correct transformed normals and view vectors from the vertex shader.

2. User Interaction

2.1 Key Interaction

The Arrow keys orbit and tilt the camera around the board, letting the player inspect the board and dice from any viewing angle without losing focus on the center of the scene. W and S zoom the camera in and out along its current view direction. The Space bar triggers the dice roll animation described above, giving the player a simple, discoverable control scheme with no on-screen UI required.

2.2 Mouse Interaction

Clicking on the board cycles through its five available skins — the default image texture (ludo.jpg) plus four canvas-generated procedural themes (Classic, Neon, Pastel and Walnut) — by swapping the shader material's map uniform and, for the procedural themes, regenerating the canvas texture with a different color palette. This lets the player change the visual theme of the scene instantly.

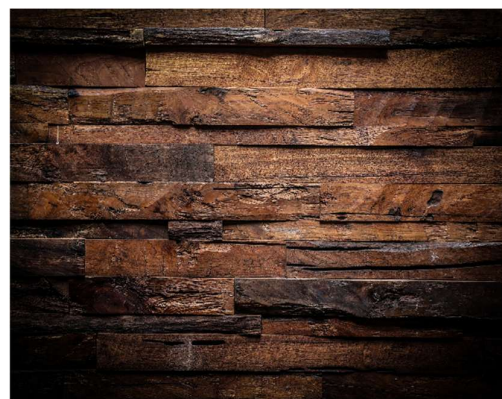
3. Design and Aesthetic Aspects

3.1 Visual Elements

The board's default look and the table surface both come from externally loaded images (ludo.jpg and background.png respectively). Everything else is generated procedurally at runtime with canvas drawing: the four alternate board skins and the dice pip faces. The lighting model is a hand-written Blinn-Phong shader (rather than a built-in MeshPhongMaterial/MeshStandardMaterial) for the board and dice, so that the specular highlight, fresnel rim glow and a subtle animated pulse can be tuned precisely; the same light values used elsewhere in the scene are fed into this shader as uniforms so the board and dice react to the ambient, directional and point lights exactly as the rest of the scene does. The decorative pawns and table use Three.js's standard PBR material instead, giving them a slightly different but complementary look.

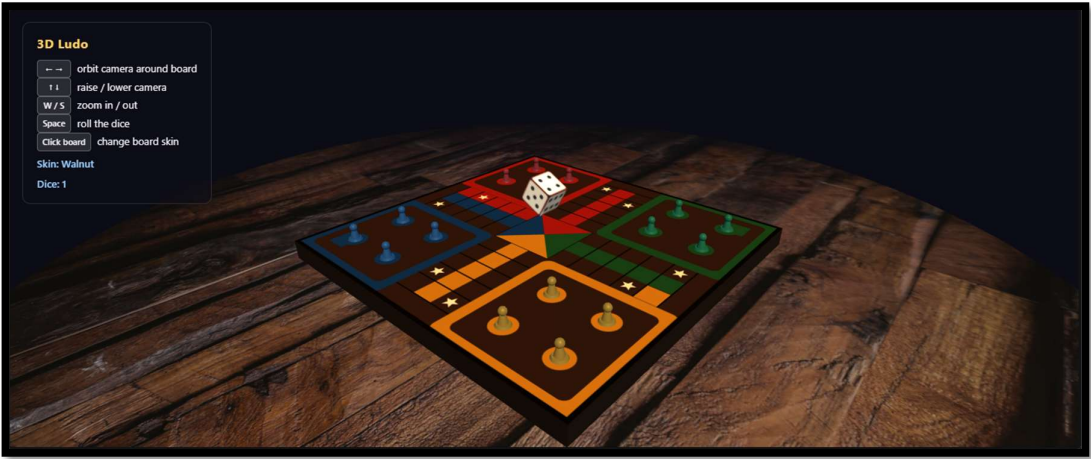


3.1.1 : ludo.jpg

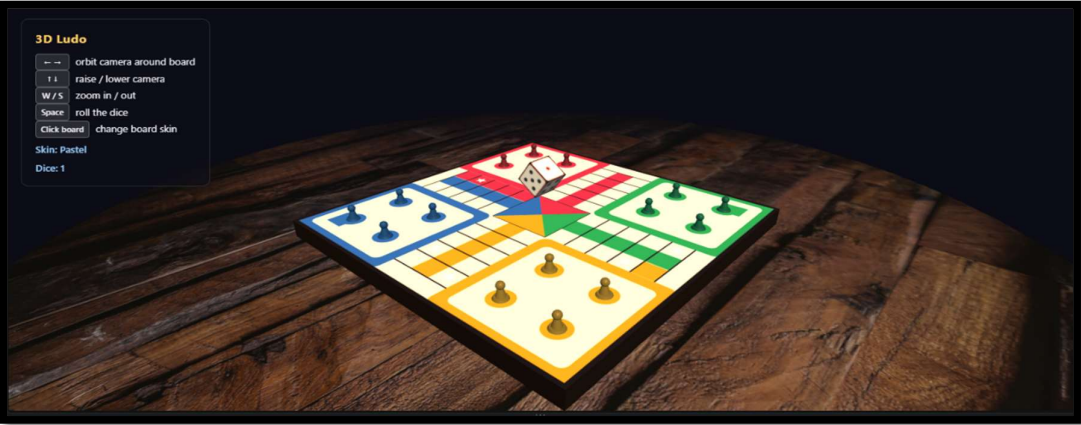


3.1.2 : background.png

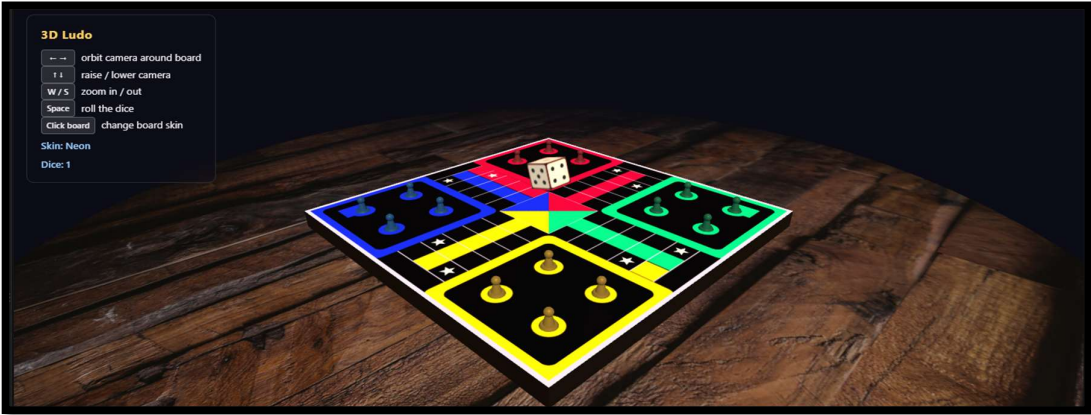
3.2 Snapshots



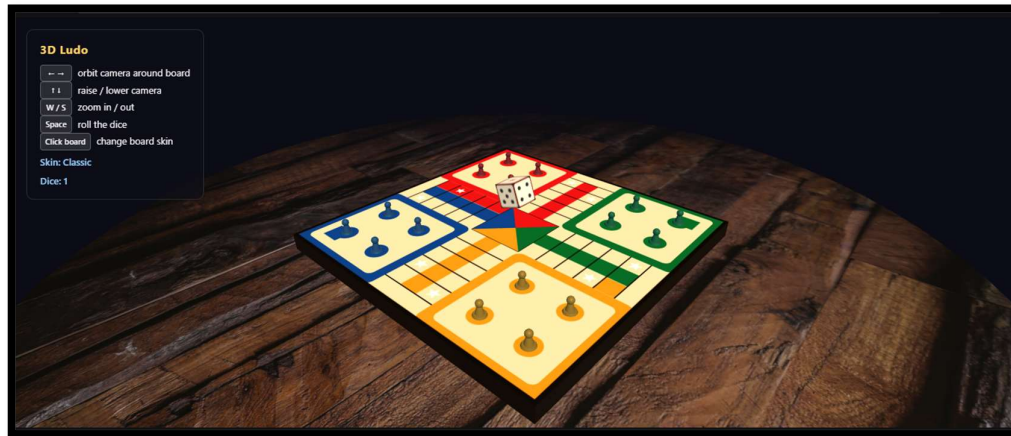
3.2.1 : Board Skin Walnut



3.2.2 : Board Skin Pastel



3.2.3 : Board Skin Neon



3.2.4 : Board Skin Classic



3.2.5 : Board Skin Bengali Ludo

[the default board view, each of the five board skins after clicking, the dice mid-tumble during a roll,

and a close-up of the custom shader's specular/fresnel highlight.]

4. Code Implementation

4.1 Software Platform

HTML, CSS, JavaScript (ES modules), Three.js, WebGL, GLSL. Three.js is loaded via an import map, so no bundler or npm install step is required — the project runs from a local static server. Two image assets are loaded at runtime with `THREE.TextureLoader`: `ludo.jpg` for the board's default skin and `background.png` for the table surface; every other texture is generated with the HTML Canvas API.

4.2 Custom Shaders

litVertexShader passes the world-space position, transformed normal and UV coordinates to the fragment stage for both the board and the dice. boardFragmentShader and diceFragmentShader each implement Blinn-Phong shading manually: ambient, diffuse and specular contributions are computed per-light from the uniforms mirroring the scene's AmbientLight, DirectionalLight and PointLight, a Fresnel term adds a soft rim highlight at grazing angles, and a small time-driven pulse uniform animates the highlight intensity subtly over time. Both shaders sample a swappable map uniform, which is how the board's texture can be switched between the loaded image and the procedural canvas skins without rebuilding the material.

4.3 Project Feature Table:

#	Features	Status
1	Custom shaders (Blinn-Phong lighting, specular, fresnel, animated pulse)	Implemented
2	Lighting (Ambient, shadow-casting Directional, Point accent) fed to shaders	Implemented
3	Perspective projection (THREE.PerspectiveCamera)	Implemented
4	Texture per object (board: image texture + 4 canvas-procedural skins; table: image texture; dice pips: canvas-procedural)	Implemented
5	Animation (idle dice spin + tumble-to-settle roll)	Implemented
6	Keyboard interaction (Arrow keys orbit/tilt, W/S zoom, Space rolls dice)	Implemented
7	Mouse interaction (click board to cycle 5 skins: image + 4 procedural)	Implemented

Table 01: Project Feature Table

5. Contribution

Student	Contribution
Sharadindu Dutta (20220104037)	[Fill in — e.g. Three.js scene setup, custom shaders, dice roll animation, keyboard/mouse interaction logic]
Kalvin Chakma (20220104038)	[Fill in — e.g. board texture generation, skin variants, lighting setup, report writing]